TwinMind - Live Suggestions Assignment

About TwinMind

TwinMind is an always-on AI meeting copilot used by people across many countries. We are scaling to millions of users. This assignment evaluates the single most important capability in our product: showing the right thing at the right time, while a conversation is happening.

We evaluate your prompt engineering skill, your full-stack engineering ability, and your code quality.

What you will build

A web app that listens to live audio from the user's mic and continuously surfaces 3 useful suggestions based on what is being said. Clicking a suggestion opens a detailed answer in a chat panel on the right.

Reference UI/UX prototype:

Open the prototype, click the mic, and you will see exactly what we expect — transcript on the left, live suggestions in the middle, chat on the right. Build to this layout. Spend your time on prompts, context, model choice, latency, and clean code — not UI exploration.

Also, download TwinMind and use the live suggestions feature yourself before you start. Your job is to improve on it. Submissions from candidates who clearly have not used the product are an immediate flag.

Functional requirements

Mic + transcript (left column)

- Start/stop mic button.
- Transcript appends in chunks roughly every 30 seconds while recording.
- Auto-scrolls to the latest line.

Live suggestions (middle column)

- Transcript and suggestions refreshes automatically every ~30 seconds
- A refresh button which manually updates transcript then suggestions if tapped.
- Each refresh produces exactly 3 fresh suggestions based on recent transcript context.
- New batch of 3 suggestions appears at the top; older batches stay visible below.
- Each suggestion is a tappable card with a short, useful preview. The preview alone should already deliver value even if not clicked. Clicking provides even more useful details.
- Suggestions should be different based on context: it could be a question to ask, a talking point, an answer to a question just asked, fact-checking a statement that was said, or clarifying info. The 3 suggestions can be a mix of the above. You decide what makes sense when. Showing the right mix of suggestions at the right time based on context is what we will be judging.

Chat (right column)

- Clicking a suggestion adds it to the chat and returns a detailed answer (separate, longer-form prompt with full transcript context).
- Users can also type questions directly.
- One continuous chat per session. No login, no data persistence needed when reloading the page.

Export

- A button to export the full session: transcript + every suggestion batch + full chat history with timestamps for each. JSON or plain text is fine. We use this to evaluate submissions.

Technical Requirements:

- **Models:** Groq for everything. Whisper Large V3 for transcription. GPT-OSS 120B for suggestions and chat. Same model for everyone so we are comparing prompts quality.
- **API key:** Settings screen where the user pastes their own Groq API key. Do not hard-code or ship a key.
- **Settings:** Editable fields for modifying the prompts and other settings: live suggestion prompt, detailed answers on-click prompt, chat prompt, context window for live suggestions, context window for expanded answers on click, etc. Hardcode the default parameters with the optimal values and prompts that you find
- **Hosting:** Deploy yourself — Vercel, Replit, Netlify, anything. Submit a public URL along with your github code.

What we evaluate

In rough priority order:

1. Quality of live suggestions: Useful, well-timed, varied by context.
2. Quality of detailed chat answers when clicked.
3. Prompt engineering: What context you pass, how much, how you structure it, how you decide when and what suggestions to surface, overall experience and value a user gets when using your product in different types of meetings.
4. Full-stack engineering: Frontend polish, backend structure, audio capture and chunking, API integration, error handling.
5. Code quality: Clean structure, readable code, sensible abstractions, no dead code, useful README.
6. Latency: Reload click to suggestions rendered, and chat sent to the first token.
7. Overall experience: Does it feel responsive and trustworthy during a real conversation?

Deliverables

- **Deployed web app URL**: Public, openable in a browser, working end-to-end once a Groq API key is pasted in.

- **GitHub repository link**: Public or shared with us. Include a README covering setup, stack choices, prompt strategy, and tradeoffs. We will read the code.

Submission

- **Deadline: 10 days from receipt.**
- During your interview, you will share your screen, open your deployed app, and we will use it live while we talk.

Notes

- Do not over-engineer. We are not evaluating production-readiness at scale. We are evaluating whether you can prompt an AI to make it useful in real time, and whether the code is something we would want in our codebase.
- If you are uncertain about a UX detail, follow the prototype.
- If you are uncertain about a prompt-engineering decision, that is the assignment — make the call and be ready to defend it.